The ReAct Pattern & LangChain Agents

1. Why Agents Need Reasoning

Having good tools is only the first step. In real-world applications, users ask agents to handle complex, multi-step tasks such as research, planning, data analysis, or automation testing workflows. A basic tool-calling system often fails when faced with ambiguity or tasks requiring multiple actions.

Agents need the ability to think step by step, decide what to do next, learn from tool outputs, and adjust their approach. The ReAct pattern provides a structured way to achieve this intelligent behavior.

---

2. The ReAct Pattern Explained

ReAct stands for **Reason + Act**. It is one of the most popular and effective frameworks for building reliable agents.

Instead of letting the LLM generate a final answer immediately, the ReAct pattern forces the model to follow a repeating cycle: it first produces a **Thought** (internal reasoning), then decides on an **Action** (tool call), receives an **Observation** (tool result), and uses that information to generate the next Thought.

Basic ReAct Flow Example:

```
Thought: I need to explore the dead letter queue first.
Action: explore_dead_letters(queue_name="payment_queue")
Observation: Found 12 dead letters...
Thought: Now I should summarize the errors.
Action: summarize_errors(...)
```

---

3. ReAct Loop in Detail

The ReAct loop typically follows these four steps repeatedly:

1. **Thought**: The agent analyzes the current situation.
2. **Action**: Calls one or more tools.
3. **Observation**: Receives tool results.
4. **Next Thought**: Reflects and plans the next step.

This cycle continues until the agent generates a final answer.

LangChain Implementation Snippet:

```
from langchain.agents import create_react_agent, AgentExecutor

agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

---

4. Thinking Models vs Normal Models in ReAct

Traditional LLMs can use the ReAct pattern, but newer "thinking" models perform much better.

These models are trained to do extended internal reasoning, which makes the Thought steps more coherent.

Example of using a thinking model:

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="o1-mini")        # or "claude-3-5-sonnet"
# or Grok, etc.
```

---

5. LangChain Agents Basics

LangChain provides a high-level abstraction called **Agents** that makes implementing ReAct easy.

The `AgentExecutor` manages the entire reasoning loop automatically.

Basic Agent Creation:

```
from langchain.agents import create_agent

agent = create_agent(
    llm=llm,
    tools=tools,
    prompt=system_prompt,
    memory=memory
)
```

---

6. Creating Agents with create_agent

The simplest way is using `create_agent()` :

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant..."),
    ("placeholder", "{chat_history}"),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

---

7. Best Practices for Building Agents

Key practices include:

- Clear tool design
- Structured outputs with Pydantic
- Parallel tool execution

Parallel Tool Example:

```
import asyncio
from langchain_core.tools import tool

@tool
async def tool1(...):
    ...

results = await asyncio.gather(
    tool1.invoke(...),
    tool2.invoke(...)
)
```

---

8. LangGraph vs Classic LangChain Agents

Classic agents are simple and fast, while **LangGraph** offers more control with custom graphs and persistent state.

```
# LangGraph example (basic)
from langgraph.graph import StateGraph

graph = StateGraph(state_schema)
# Add nodes and edges...
```

---

9. Common Pitfalls & Real-World Tips

Common issues:

- Infinite loops → Set `max_iterations`
- Bad tool selection → Improve docstrings
- High cost → Use cheaper models for simple steps

Tip for debugging:

```
agent_executor = AgentExecutor(..., verbose=True, max_iterations=10)
```
